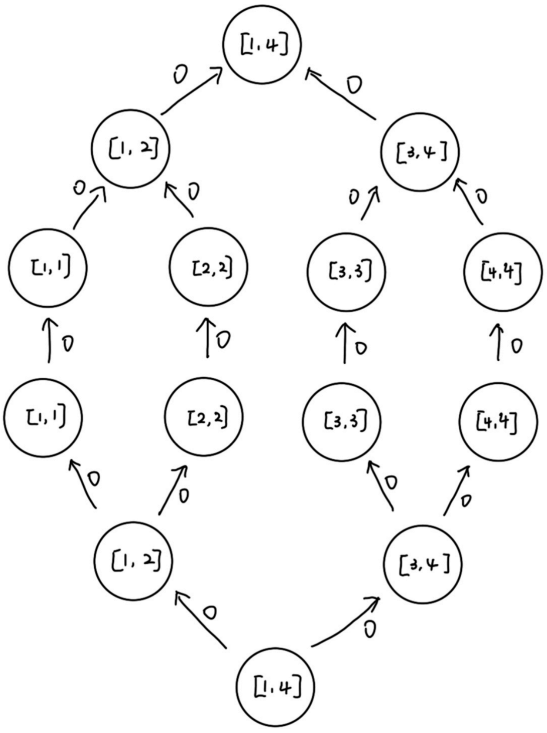
区間に辺を張るテク

update 2026.4.19: 更新了线段树加速建图下的2-SAT 来自2025ICPC武汉B

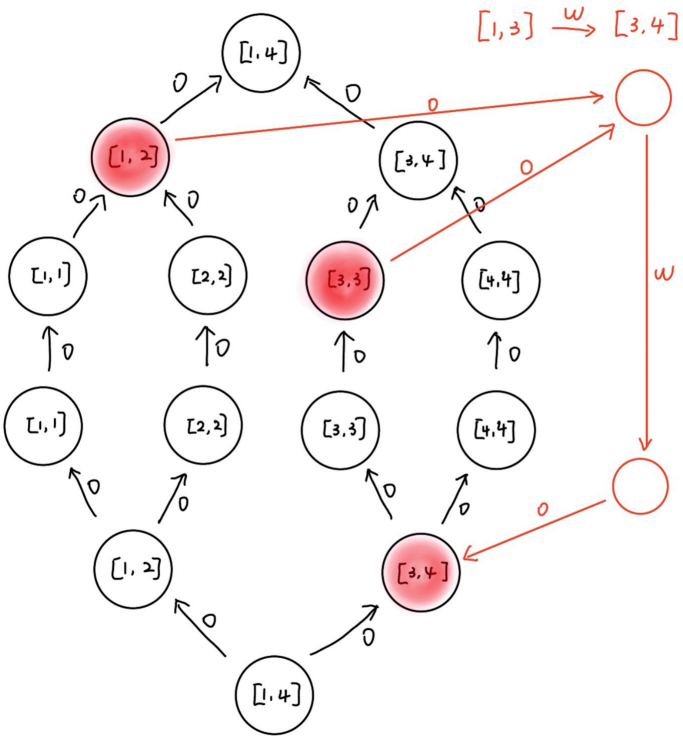
基于线段树拉伸边的最短路问题

用于处理一类特殊的最短路问题: 加边的方式如下, 对 $\forall u, v \in [l, r]$, 添加边 (u, v, w) , 即两个区间内的点两两连接相同权重的边.

首先是建图, 基于线段树将区间 $[1, n]$ 拆分成 $\log n$ 个区间, 并分成上下两部分, 当没有连边时, 建图如下:



当区间 $[1, 3]$ 向区间 $[3, 4]$ 连权值为 w 的边时, 建虚点:



例题: NIKKEI Programming Contest 2019-2 D - Shortest Path on a Line 、Codeforces Round 406(Div.1) B. Legacy

进阶: AtCoder Beginner Contest 414 G - AtCoder Express 4

Shortest Path on a Line

题意: 给出 n 个点的无向图, 起初图没有边, 进行 m 次操作, 每次操作给出 $[L, R, C]$, 对 $\forall s, t$ 满足 $L \le s < t \le R$, s 向 t 连权值为 w 的无向边. 求最终 1 到 n 的最短路.

```
struct segment_dijkstra {
    typedef pair<int,int> pii;
    const int inf = 0x3f3f3f3f3f3f3f3f;
    struct node {
        int l, r;
        int fid, gid;
    };
    int n, maxn, idx;
    vector<node> tr;
    vector<vector<pii>> e;
    vector<int> idf, idg;

    void add(int u, int v, int w) {
        e[u].push_back({v, w});
    }

```

```
    void build(int u, int l, int r) {
        tr[u] = {l, r};
        tr[u].fid = ++ idx, tr[u].gid = ++ idx;
        if(l == r) {
            idf[l] = tr[u].fid, idg[l] = tr[u].gid;
            add(tr[u].gid, tr[u].fid, 0);
            return;
        }
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
        add(tr[u << 1].fid, tr[u].fid, 0), add(tr[u << 1 | 1].fid, tr[u].fid, 0);
        add(tr[u].gid, tr[u << 1].gid, 0), add(tr[u].gid, tr[u << 1 | 1].gid, 0);
    }

    // 点数, 操作数
    segment_dijkstra(int _n, int _m) {
        n = _n, maxn = _n * 8 + _m * 2;
        tr.resize(n * 4 + 10);
        e.resize(maxn + 10);
        idf.resize(n + 1), idg.resize(n + 1);
        idx = 0;
        build(1, 1, n);
    }

    vector<int> query(int u, int l, int r, int op) {
        vector<int> res;
        if(tr[u].l >= l && tr[u].r <= r) {
            if(op == 0) res.push_back(tr[u].fid);
            else res.push_back(tr[u].gid);
            return res;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) {
            vector<int> t = query(u << 1, l, r, op);
            for(auto u: t) res.push_back(u);
        }
        if(r > mid) {
            vector<int> t = query(u << 1 | 1, l, r, op);
            for(auto u: t) res.push_back(u);
        }
        return res;
    }

    void modify(int l1, int r1, int l2, int r2, int w) {
        vector<int> f = query(1, l1, r1, 0);
```

```
        vector<int> g = query(1, l2, r2, 1);
        int nf = ++ idx, ng = ++ idx;
        for(auto u: f) add(u, nf, 0);
        add(nf, ng, w);
        for(auto u: g) add(ng, u, 0);
    }

    int dijkstra() {
        vector<int> dis(idx + 1, inf);
        vector<int> vis(idx + 1, 0);
        priority_queue<pii,vector<pii>,greater<pii>> q;
        dis[idf[1]] = 0;
        q.push({0, idf[1]});

        while(q.size()) {
            auto t = q.top();
            q.pop();
            int d = t.first, u = t.second;
            if(vis[u]) continue;
            vis[u] = 1;
            for(auto [v, w]: e[u])
                if(dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    q.push({dis[v], v});
                }
        }
        return dis[idg[n]] > inf / 2 ? -1 : dis[idg[n]];
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    segment_dijkstra Tr(n, m);
    while(m --) {
        int l, r, w;
        cin >> l >> r >> w;
        Tr.modify(l, r, l, r, w);
    }
    cout << Tr.dijkstra() << '\n';
}
```

Legacy

题意: 给出 n 个点 m 个操作以及起点 s , 操作共有三种: u 向 v 连权值为 w 的边, u 向区间 $[l, r]$ 连权值为 w 的边, 区间 $[l, r]$ 向 u 连权值为 w 的边, 求从起点 s 出发到每个点的最短路.

```
struct segment_dijkstra {
    typedef pair<int,int> pii;
    const int inf = 0x3f3f3f3f3f3f3f3f;
    struct node {
        int l, r;
        int fid, gid;
    };
    int n, maxn, idx;
    vector<node> tr;
    vector<vector<pii>> e;
    vector<int> idf, idg;

    void add(int u, int v, int w) {
        e[u].push_back({v, w});
    }

    void build(int u, int l, int r) {
        tr[u] = {l, r};
        tr[u].fid = ++ idx, tr[u].gid = ++ idx;
        if(l == r) {
            idf[l] = tr[u].fid, idg[l] = tr[u].gid;
            add(tr[u].gid, tr[u].fid, 0);
            return;
        }
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
        add(tr[u << 1].fid, tr[u].fid, 0), add(tr[u << 1 | 1].fid, tr[u].fid, 0);
        add(tr[u].gid, tr[u << 1].gid, 0), add(tr[u].gid, tr[u << 1 | 1].gid, 0);
    }

    // 点数, 操作数
    segment_dijkstra(int _n, int _m) {
        n = _n, maxn = _n * 8 + _m * 2;
        tr.resize(n * 4 + 10);
        e.resize(maxn + 10);
        idf.resize(n + 1), idg.resize(n + 1);
        idx = 0;
        build(1, 1, n);
    }

```

```
    vector<int> query(int u, int l, int r, int op) {
        vector<int> res;
        if(tr[u].l >= l && tr[u].r <= r) {
            if(op == 0) res.push_back(tr[u].fid);
            else res.push_back(tr[u].gid);
            return res;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) {
            vector<int> t = query(u << 1, l, r, op);
            for(auto u: t) res.push_back(u);
        }
        if(r > mid) {
            vector<int> t = query(u << 1 | 1, l, r, op);
            for(auto u: t) res.push_back(u);
        }
        return res;
    }

    void modify(int l1, int r1, int l2, int r2, int w) {
        vector<int> f = query(1, l1, r1, 0);
        vector<int> g = query(1, l2, r2, 1);
        int nf = ++ idx, ng = ++ idx;
        for(auto u: f) add(u, nf, 0);
        add(nf, ng, w);
        for(auto u: g) add(ng, u, 0);
    }

    vector<int> dijkstra(int s) {
        vector<int> dis(idx + 1, inf);
        vector<int> vis(idx + 1, 0);
        priority_queue<pii,vector<pii>,greater<pii>> q;
        dis[idf[s]] = 0;
        q.push({0, idf[s]});

        while(q.size()) {
            auto t = q.top();
            q.pop();
            int d = t.first, u = t.second;
            if(vis[u]) continue;
            vis[u] = 1;
            for(auto [v, w]: e[u])
                if(dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    q.push({dis[v], v});
                }
        }
```

```
        vector<int> res(n + 1);
        for(int i = 1; i <= n; i++)
            res[i] = dis[idg[i]] > inf / 2 ? -1 : dis[idg[i]];
        return res;
    }
};

void solve() {
    int n, m, s;
    cin >> n >> m >> s;
    segment_dijkstra Tr(n, m);
    while(m --) {
        int op;
        cin >> op;
        if(op == 1) {
            int u, v, w;
            cin >> u >> v >> w;
            Tr.modify(u, u, v, v, w);
        }
        if(op == 2) {
            int u, l, r, w;
            cin >> u >> l >> r >> w;
            Tr.modify(u, u, l, r, w);
        }
        if(op == 3) {
            int u, l, r, w;
            cin >> u >> l >> r >> w;
            Tr.modify(l, r, u, u, w);
        }
    }
    vector<int> res = Tr.dijkstra(s);
    res[s] = 0;
    for(int i = 1; i <= n; i++)
        cout << res[i] << " \n"[i == n];
}
```

AtCoder Express 4

题意: 有 n 个车站和 m 条线路, 每条线路能从 $[l_1, r_1]$ 上车, 在 $[l_2, r_2]$ 下车, 满足 $l_1 \le r_1 < l_2 \le r_2$ 或 $l_2 \le r_2 < l_1 \le r_1$, 从 s 到 t 的代价为 $|x[s] - x[t]| + c$, 求 1 到每个点的最短路

思路: 以向右行驶的线路为例, 对于一条线路 ($[l_1, r_1]$ 上车, $[l_2, r_2]$ 下车, 起步价为 c), 建两个虚点 u , v . 用 u 聚合上车路段的顶点, 顶点对应路段为 $[a, b]$ 则顶点向 u 连权值为 $x[r_1] - x[b]$ 的边, 用 v 聚合下车路段的顶点, 顶点对应路线为 $[a, b]$ 则 v 向顶点连权值为 $x[a] - x[l_2]$ 的边, u 向 v 连权值为 c 的边

```cpp
int n, m, p[N];
int idx;
vector<pii> e[M];

void add(int u, int v, int w) {
    e[u].push_back({v, w});
}

struct node {
    int l, r;
    int L_fid, L_gid;   // 向左列车的上下节点id
    int R_fid, R_gid;   // 向右列车的上下节点id
}tr[N * 4];

void build(int u, int l, int r) {
    tr[u] = {l, r};
    tr[u].L_fid = ++ idx, tr[u].L_gid = ++ idx;
    tr[u].R_fid = ++ idx, tr[u].R_gid = ++ idx;
    if(l == r) {
        // gid -> l -> fid
        add(tr[u].L_gid, l, 0), add(l, tr[u].L_fid, 0);
        add(tr[u].R_gid, l, 0), add(l, tr[u].R_fid, 0);
        return;
    }
    int mid = l + r >> 1;
    build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
    add(tr[u << 1].L_fid, tr[u].L_fid, p[tr[u << 1].l] - p[tr[u].l]);
    add(tr[u << 1 | 1].L_fid, tr[u].L_fid, p[tr[u << 1 | 1].l] - p[tr[u].l]);
    add(tr[u].L_gid, tr[u << 1].L_gid, p[tr[u].r] - p[tr[u << 1].r]);
    add(tr[u].L_gid, tr[u << 1 | 1].L_gid, p[tr[u].r] - p[tr[u << 1 | 1].r]);
    add(tr[u << 1].R_fid, tr[u].R_fid, p[tr[u].r] - p[tr[u << 1].r]);
    add(tr[u << 1 | 1].R_fid, tr[u].R_fid, p[tr[u].r] - p[tr[u << 1 | 1].r]);
    add(tr[u].R_gid, tr[u << 1].R_gid, p[tr[u << 1].l] - p[tr[u].l]);
    add(tr[u].R_gid, tr[u << 1 | 1].R_gid, p[tr[u << 1 | 1].l] - p[tr[u].l]);
}

// op1: 0L, 1R; op2: 0f, 1g; op3: 需要的信息 0l, 1r
// 返回 pii : {id, 所需端点}
vector<pii> query(int u, int l, int r, int op1, int op2, int op3) {
    vector<pii> res;
    if(tr[u].l >= l && tr[u].r <= r) {
        int inf1, inf2;
        if(op1 == 0 && op2 == 0)
            inf1 = tr[u].L_fid;
        if(op1 == 0 && op2 == 1)
            inf1 = tr[u].L_gid;
        if(op1 == 1 && op2 == 0)
```

```
            inf1 = tr[u].R_fid;
        if(op1 == 1 && op2 == 1)
            inf1 = tr[u].R_gid;
        if(op3 == 0) inf2 = tr[u].l;
        else inf2 = tr[u].r;
        res.push_back({inf1, inf2});
        return res;
    }
    int mid = tr[u].l + tr[u].r >> 1;
    if(l <= mid) {
        vector<pii> t = query(u << 1, l, r, op1, op2, op3);
        for(auto u: t) res.push_back(u);
    }
    if(r > mid) {
        vector<pii> t = query(u << 1 | 1, l, r, op1, op2, op3);
        for(auto u: t) res.push_back(u);
    }
    return res;
}

void solve() {
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
        cin >> p[i];

    idx = n;
    build(1, 1, n);

    while(m --) {
        int l1, r1, l2, r2, c;
        cin >> l1 >> r1 >> l2 >> r2 >> c;
        if(l1 > r2) {   // to L
            vector<pii> f = query(1, l1, r1, 0, 0, 0);
            vector<pii> g = query(1, l2, r2, 0, 1, 1);
            int nf = ++ idx, ng = ++ idx;
            for(auto [id, l]: f) add(id, nf, p[l] - p[l1]);
            add(nf, ng, p[l1] - p[r2] + c);
            for(auto [id, r]: g) add(ng, id, p[r2] - p[r]);
        } else {    // to R
            vector<pii> f = query(1, l1, r1, 1, 0, 1);
            vector<pii> g = query(1, l2, r2, 1, 1, 0);
            int nf = ++ idx, ng = ++ idx;
            for(auto [id, r]: f) add(id, nf, p[r1] - p[r]);
            add(nf, ng, p[l2] - p[r1] + c);
            for(auto [id, l]: g) add(ng, id, p[l] - p[l2]);
        }
    }
```

```

    vector<int> dis(idx + 1, inf);
    vector<int> vis(idx + 1, 0);
    priority_queue<pii,vector<pii>,greater<pii>> q;
    dis[1] = 0;
    q.push({0, 1});
    while(q.size()) {
        auto t = q.top();
        q.pop();
        int d = t.first, u = t.second;
        if(vis[u]) continue;
        vis[u] = 1;
        for(auto [v, w]: e[u])
            if(dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                q.push({dis[v], v});
            }
    }
    for(int i = 2; i <= n; i++)
        cout << (dis[i] > inf / 2 ? -1 : dis[i]) << " \n"[i == n];
}
```

77G Network

题意：有一棵以 1 为根的树，共 n 个节点。要在若干城市安装基站，要求每条边的两个端点至少有一个装基站。同时有 m 个故障约束：若在节点 x 装基站，则其子树中与 x 距离恰好为 y 的那些节点都不能装基站（若子树中无此类节点可忽略）。对每组数据，判断是否存在满足所有约束的选择，并给出一个合法集合或输出 "No"

数据范围：$1 \le T \le 10^3$，$2 \le n \le 2 \times 10^5$，$0 \le m \le 2 \times 10^5$，$\sum n \le 2 \times 10^5$，$\sum m \le 2 \times 10^5$

题解：把每个城市 u 的选择视为一个布尔变量 X_u（真 = 装基站）。题目两类约束都能写成两个字面的合取字句：边约束：每条边 (u, v) 要求 $X_u \vee X_v$；故障约束：若 X_u 为真，则一批节点 v 必须为假，等价于对每个相关 v 加入子句 $\neg X_u \vee \neg X_v$。因此整个问题可以转化为 $2 - SAT$ 问题。直接暴力建图在最坏情况下会产生 $O(n^2)$ 的边，无法接受。把所有深度相同的节点按 dfs 序排序后，线段树建图即可

```
struct segment_two_sat {
    typedef pair<int,int> pii;
    const int inf = 0x3f3f3f3f3f3f3f3f;
    struct node {
        int l, r;
        int fid, gid;
    };
    int n, maxn, idx;
```

```
    vector<node> tr;
    vector<vector<int>> e;
    vector<int> idf, idg;

    void add(int u, int v) {
        e[u].push_back(v);
    }

    void build(int u, int l, int r) {
        tr[u] = {l, r};
        tr[u].fid = ++ idx, tr[u].gid = ++ idx;
        if(l == r) {
            idf[l] = tr[u].fid, idg[l] = tr[u].gid;
            add(tr[u].gid, tr[u].fid);
            return;
        }
        int mid = l + r >> 1;
        build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
        add(tr[u << 1].fid, tr[u].fid), add(tr[u << 1 | 1].fid, tr[u].fid);
        add(tr[u].gid, tr[u << 1].gid), add(tr[u].gid, tr[u << 1 | 1].gid);
    }

    // 点数, 操作数
    segment_two_sat(int _n, int _m) {
        n = _n, maxn = _n * 8 + _m * 2;
        tr.resize(n * 4 + 10);
        e.resize(maxn + 10);
        idf.resize(n + 1), idg.resize(n + 1);
        idx = 0;
        build(1, 1, n);
    }

    vector<int> query(int u, int l, int r, int op) {
        vector<int> res;
        if(tr[u].l >= l && tr[u].r <= r) {
            if(op == 0) res.push_back(tr[u].fid);
            else res.push_back(tr[u].gid);
            return res;
        }
        int mid = tr[u].l + tr[u].r >> 1;
        if(l <= mid) {
            vector<int> t = query(u << 1, l, r, op);
            for(auto u: t) res.push_back(u);
        }
        if(r > mid) {
            vector<int> t = query(u << 1 | 1, l, r, op);
            for(auto u: t) res.push_back(u);
```

```
        }
        return res;
    }

    void modify1(int l1, int r1, int l2, int r2) {
        vector<int> f = query(1, l1, r1, 0);
        vector<int> g = query(1, l2, r2, 1);
        int nf = ++ idx, ng = ++ idx;
        for(auto u: f) add(u, nf);
        add(nf, ng);
        for(auto u: g) add(ng, u);
    }

    void modify2(int u, int v) {
        u = idf[u], v = idg[v];
        add(u, v);
    }

    vector<int> dfn, low, stk, instk, id;
    int timestamp, top, cnt;
    void tarjan(int u) {
        dfn[u] = low[u] = ++ timestamp;
        stk[++ top] = u, instk[u] = 1;
        for(auto v: e[u]) {
            if(!dfn[v]) {
                tarjan(v);
                low[u] = min(low[u], low[v]);
            } else if(instk[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(low[u] == dfn[u]) {
            int y;
            cnt ++;
            do {
                y = stk[top --];
                instk[y] = 0;
                id[y] = cnt;
            } while(y != u);
        }
    }

    bool two_sat() {
        dfn.assign(idx + 1, 0);
        low.assign(idx + 1, 0);
        stk.assign(idx + 1, 0);
        instk.assign(idx + 1, 0);
```

```
        id.assign(idx + 1, 0);
        timestamp = top = cnt = 0;
        for(int i = 1; i <= idx; i++)
            if(!dfn[i])
                tarjan(i);
        for(int i = 1; i <= n / 2; i++)
            if(id[idg[i]] == id[idg[i + n / 2]])
                return false;
        return true;
    }

    bool check(int u) {
        return id[idg[u]] < id[idg[u + n / 2]];
    }
};

void solve() {
    int n, m;
    cin >> n;
    vector<vector<int>> e(n + 1, vector<int> ());
    for(int i = 2, x; i <= n; i++) {
        cin >> x;
        e[x].push_back(i);
    }
    cin >> m;
    segment_two_sat tr(n * 2, m);

    vector<int> dep(n + 1), mx_dep(n + 1), L(n + 1), R(n + 1);
    int dfn = 0;
    auto dfs = [&](auto&& self, int u) -> void {
        L[u] = ++ dfn, mx_dep[u] = dep[u];
        for(auto v: e[u]) {
            dep[v] = dep[u] + 1;
            self(self, v);
            mx_dep[u] = max(mx_dep[u], mx_dep[v]);
        }
        R[u] = dfn;
    };
    dep[1] = 1;
    dfs(dfs, 1);

    vector<vector<int>> p(n + 1, vector<int> ());
    for(int i = 1; i <= n; i++)
        p[dep[i]].push_back(i);
    for(int i = 1; i <= n; i++)
        sort(p[i].begin(), p[i].end(), [&](int u, int v){return L[u] <
L[v];});
```

```
    vector<int> la(n + 1), sz(n + 1), id(n + 1);
    for(int i = 1; i <= n; i++) {
        la[i] = la[i - 1] + sz[i - 1];
        sz[i] = p[i].size();
        for(int j = 0; j < sz[i]; j++)
            id[p[i][j]] = la[i] + j + 1;
    }

    for(int u = 1; u <= n; u++)
        for(auto v: e[u]) {
            // u || v
            tr.modify2(id[u] + n, id[v]);
            tr.modify2(id[v] + n, id[u]);
        }
    for(int i = 0; i < m; i++) {
        int u, d;
        cin >> u >> d;
        d += dep[u];
        if(mx_dep[u] < d)
            continue;
        int idl, idr;
        int l = 0, r = sz[d] - 1;
        while(l < r) {
            int mid = l + r >> 1;
            if(L[p[d][mid]] >= L[u]) r = mid;
            else l = mid + 1;
        }
        idl = l, r = sz[d] - 1;
        while(l < r) {
            int mid = l + r + 1 >> 1;
            if(L[p[d][mid]] <= R[u]) l = mid;
            else r = mid - 1;
        }
        idr = l;
        // v \in [idl, idr]
        // !u || !v
        u = id[u];
        idl += la[d] + 1, idr += la[d] + 1;
        tr.modify1(u, u, idl + n, idr + n);
        tr.modify1(idl, idr, u + n, u + n);
    }

    if(tr.two_sat()) {
        cout << "Yes\n";
        vector<int> res;
        for(int i = 1; i <= n; i++)
            if(tr.check(id[i]))
```

```
                res.push_back(i);
        int sz = res.size();
        cout << sz << '\n';
        for(int i = 0; i < sz; i++)
            cout << res[i] << " \n"[i + 1 == sz];
    } else cout << "No\n";
}
```